Lecture 14. Model Building

Diamonds and daily flights: building models step by step

Contents

14.1 Building models step by step **2**

14.2 Why are low-quality diamonds more expensive? **2**

14.2.1 Price and carat 2

14.2.2 A more complicated model 4

14.3 What affects the number of daily flights? **4**

14.3.1 Day of week 5

14.3.2 Seasonal Saturday effect 6

14.3.3 Time of year: an alternative approach 7

14.4 Selected exercises **8**

14.1 Building models step by step

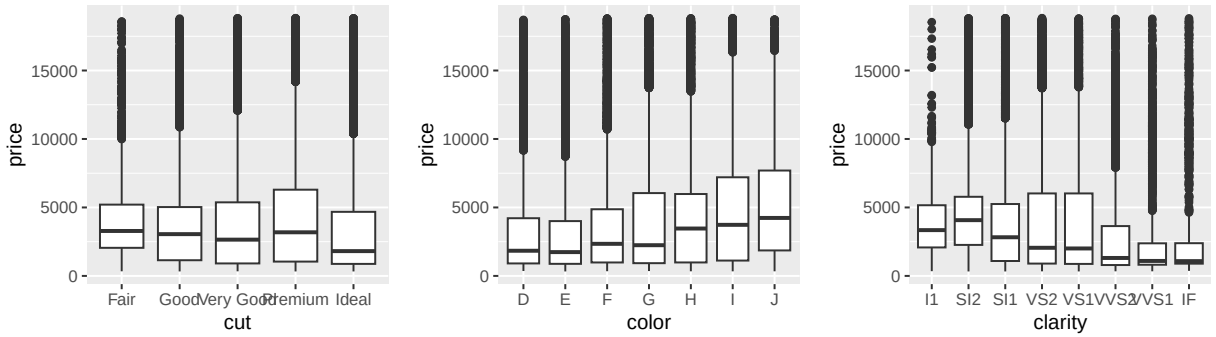
Motivation. Lecture 13 fitted models to simulated data with known structure. Real data are messier: several patterns overlap. The strategy here is to peel them off one at a time, fitting a model for the strongest pattern, looking at the residuals, and modelling what remains.

The workflow alternates between visualisation and modelling: find a pattern by plotting, make it precise with a model, then study the residuals for the next pattern. The aim is to move implicit knowledge ("prices depend on size") into an explicit, quantitative model. We use `modelr`, `nycflights13` and `lubridate`.

14.2 Why are low-quality diamonds more expensive?

Lecture 4 found that diamonds with poor cut, bad colour (J is the worst colour) and inferior clarity (I1) have higher median prices:

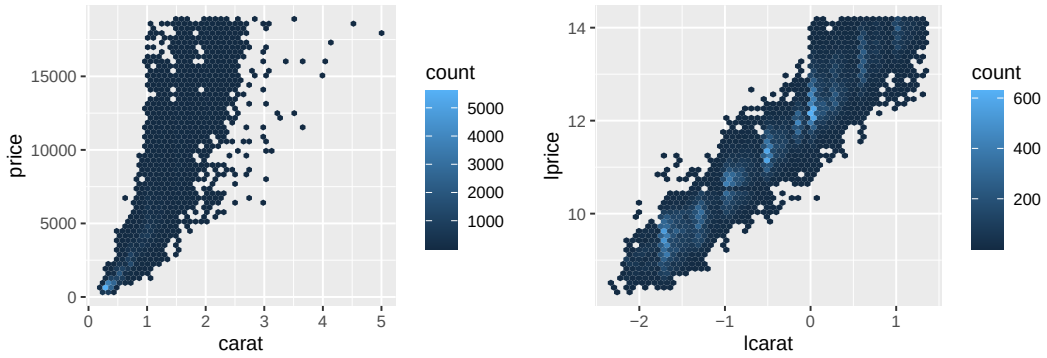
```
ggplot(diamonds, aes(cut, price)) + geom_boxplot()
ggplot(diamonds, aes(color, price)) + geom_boxplot()
ggplot(diamonds, aes(clarity, price)) + geom_boxplot()
```



14.2.1 Price and carat

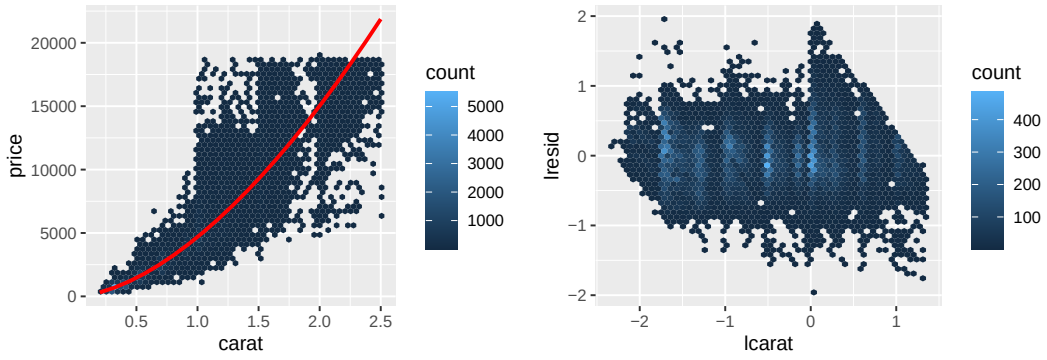
The explanation is a confounding variable: weight (`carat`) is the single most important factor in price, and low-quality diamonds tend to be larger. To see the effect of the other variables, first remove the effect of carat. Two adjustments make the relationship easier to model: keep diamonds of at most 2.5 carats (99.7% of the data) and log-transform both variables, which makes the pattern linear:

```
diamonds2 <- diamonds %>%
  filter(carat <= 2.5) %>%
  mutate(lprice = log2(price), lcarat = log2(carat))
ggplot(diamonds, aes(carat, price)) + geom_hex(bins = 50)
ggplot(diamonds2, aes(lcarat, lprice)) + geom_hex(bins = 50)
```



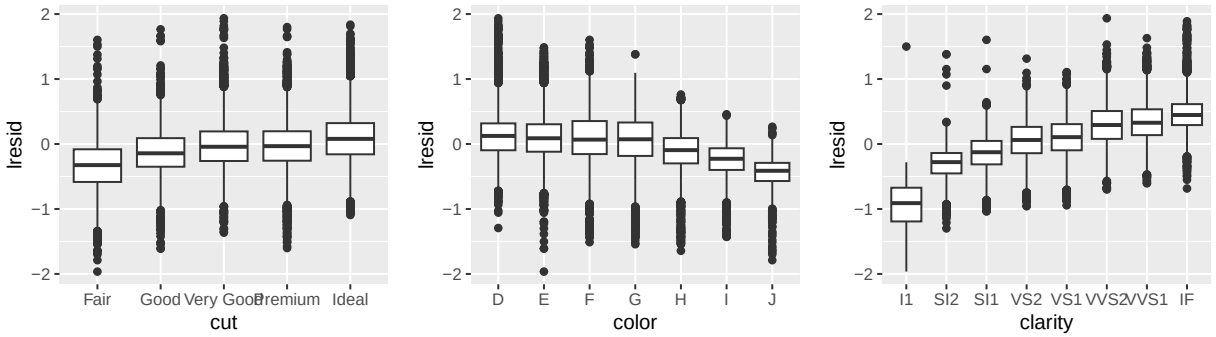
Fit a line on the log scale and back-transform the predictions to the original scale:

```
mod_diamond <- lm(lprice ~ lcarat, data = diamonds2)
grid <- diamonds2 %>%
  data_grid(carat = seq_range(carat, 20)) %>%
  mutate(lcarat = log2(carat)) %>%
  add_predictions(mod_diamond, "lprice") %>%
  mutate(price = 2 ^ lprice)
ggplot(diamonds2, aes(carat, price)) +
  geom_hex(bins = 50) +
  geom_line(data = grid, colour = "red", linewidth = 1)
diamonds2 <- diamonds2 %>% add_residuals(mod_diamond, "lresid")
ggplot(diamonds2, aes(lcarat, lresid)) + geom_hex(bins = 50)
```



The fitted curve shows that large diamonds are cheaper than the model predicts, probably because no diamond in the data costs more than $19 000. The residuals no longer depend on carat. Plotted against the quality variables, they now show the expected order:

```
ggplot(diamonds2, aes(cut, lresid)) + geom_boxplot()
ggplot(diamonds2, aes(color, lresid)) + geom_boxplot()
ggplot(diamonds2, aes(clarity, lresid)) + geom_boxplot()
```



Because the residuals are on the $\log_2$ scale, a residual of -1 means the price is half what carat alone predicts, and $+1$ means twice. As quality improves, the relative price increases.

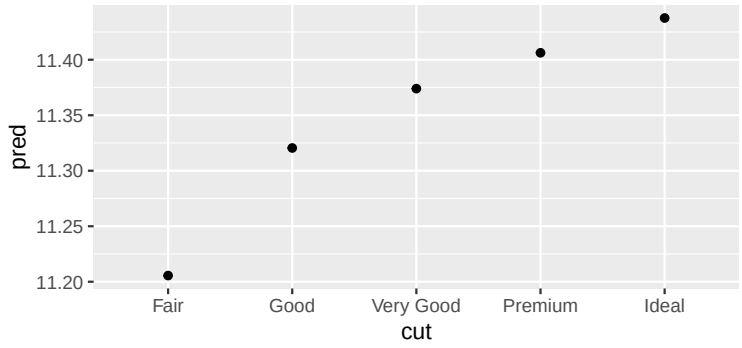
14.2.2 A more complicated model

Adding colour, cut and clarity makes their effects explicit:

```
mod_diamond2 <- lm(lprice ~ lcarat + color + cut + clarity, data = diamonds2)
```

With four predictors the model is hard to plot directly, but the effects can be plotted one at a time: `data_grid(cut, .model = mod_diamond2)` fills the other predictors with "typical" values (the median or most common level):

```
grid <- diamonds2 %>%
  data_grid(cut, .model = mod_diamond2) %>%
  add_predictions(mod_diamond2)
ggplot(grid, aes(cut, pred)) + geom_point()
```



The residuals of the larger model are mostly small, but some are large ($|r| > 1$, a factor of two). Looking at them is the start of a new exploration: they may be data-entry errors, or bargains.

```
diamonds2 <- diamonds2 %>% add_residuals(mod_diamond2, "lresid2")
diamonds2 %>%
  filter(abs(lresid2) > 1) %>%
  add_predictions(mod_diamond2) %>%
  mutate(pred = round(2 ^ pred)) %>%
  select(price, pred, carat:table, x:z) %>%
  arrange(price)
```

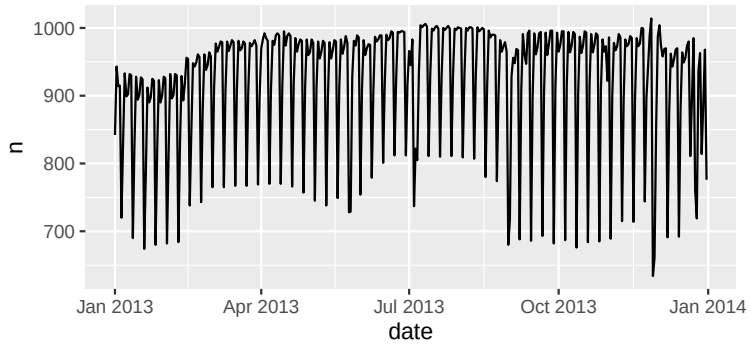
```
## # A tibble: 16 x 11
##   price  pred carat cut     color clarity depth table     x     y     z
##   <int> <dbl> <dbl> <ord>   <ord> <ord>   <dbl> <dbl> <dbl> <dbl> <dbl>
## 1  1013   264  0.25 Fair    F     SI2      54.4    64  4.3   4.23  2.32
## 2  1186   284  0.25 Premium G     SI2      59      60  5.33  5.28  3.12
## 3  1186   284  0.25 Premium G     SI2      58.8    60  5.33  5.28  3.12
## 4  1262  2644  1.03 Fair    E     I1       78.2    54  5.72  5.59  4.42
## 5  1415   639  0.35 Fair    G     VS2      65.9    54  5.57  5.53  3.66
## # i 11 more rows
```

14.3 What affects the number of daily flights?

Motivation. Counts over time mix several patterns: day-of-week cycles, seasons, holidays. Each layer is easier to see once the previous one has been modelled away.

Count the flights leaving NYC each day (365 rows):

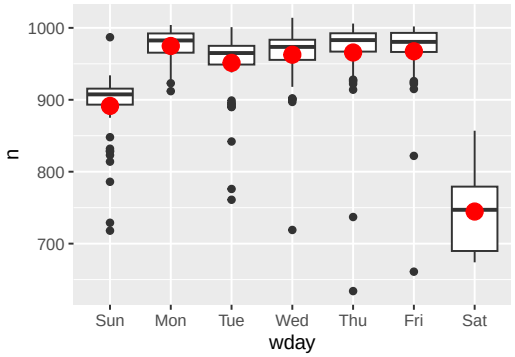
```
daily <- flights %>%
  mutate(date = make_date(year, month, day)) %>%
  group_by(date) %>%
  summarise(n = n())
ggplot(daily, aes(date, n)) + geom_line()
```

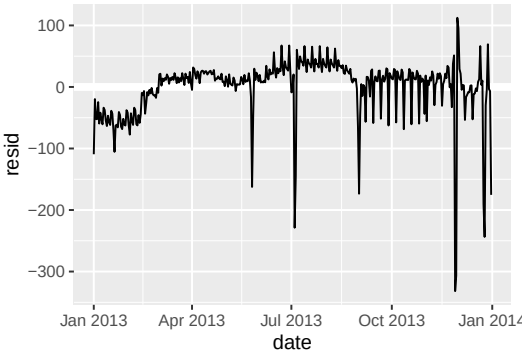


14.3.1 Day of week

The strongest pattern is weekly: most travel is for business, so there are fewer flights at weekends, especially Saturdays. A model with `wday` predicts the average per weekday:

```
daily <- daily %>% mutate(wday = wday(date, label = TRUE))
mod <- lm(n ~ wday, data = daily)
grid <- daily %>% data_grid(wday) %>% add_predictions(mod, "n")
ggplot(daily, aes(wday, n)) +
  geom_boxplot() +
  geom_point(data = grid, colour = "red", size = 4)
daily <- daily %>% add_residuals(mod)
daily %>%
  ggplot(aes(date, resid)) +
  geom_ref_line(h = 0) +
  geom_line()
```





The residuals reveal what the weekday model misses:

- from June the model fails, with a strong regular pattern left: colouring by weekday shows that Saturdays are badly fit (more Saturday flights in summer, fewer in autumn);
- some days have far fewer flights than expected; `filter(resid < -100)` identifies US holidays (New Year, 4 July, Thanksgiving, Christmas);
- a smooth long-term trend remains, fewer flights in January and December, more in summer (`geom_smooth(span = 0.20)`).

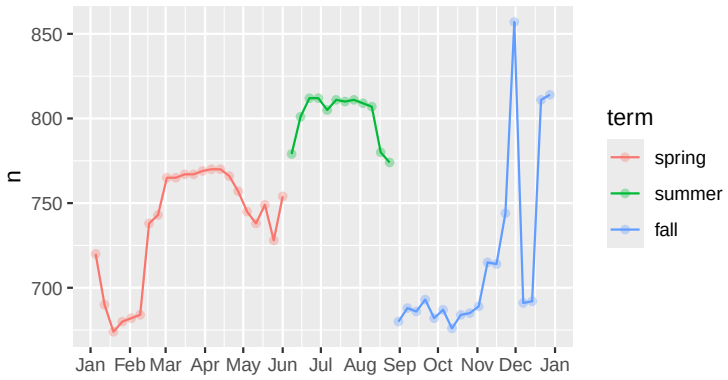
```
daily %>% filter(resid < -100)
```

```
## # A tibble: 11 x 4
##   date           n wday  resid
##   <date>     <int> <ord> <dbl>
## 1 2013-01-01   842 Tue   -109.
## 2 2013-01-20   786 Sun   -105.
## 3 2013-05-26   729 Sun   -162.
## 4 2013-07-04   737 Thu   -229.
## 5 2013-07-05   822 Fri   -145.
## # i 6 more rows
```

14.3.2 Seasonal Saturday effect

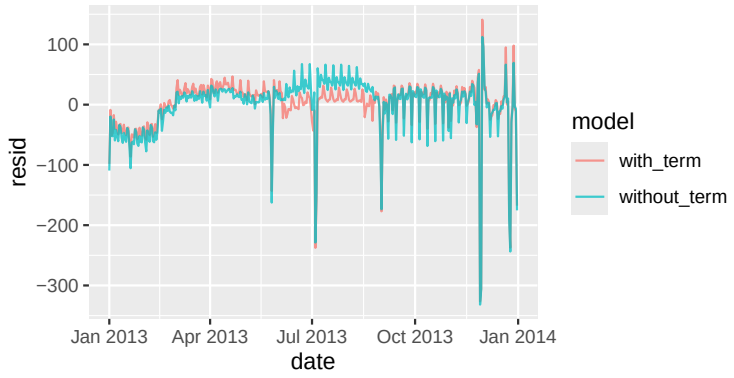
Plotting Saturdays alone suggests a school-term effect: people travel on Saturdays in summer holidays. A `term` variable with breaks chosen from the plot captures it:

```
term <- function(date) {
  cut(date,
      breaks = ymd(20130101, 20130605, 20130825, 20140101),
      labels = c("spring", "summer", "fall"))
}
daily <- daily %>% mutate(term = term(date))
daily %>%
  filter(wday == "Sat") %>%
  ggplot(aes(date, n, colour = term)) +
  geom_point(alpha = 1/3) +
  geom_line() +
  scale_x_date(NULL, date_breaks = "1 month", date_labels = "%b")
```



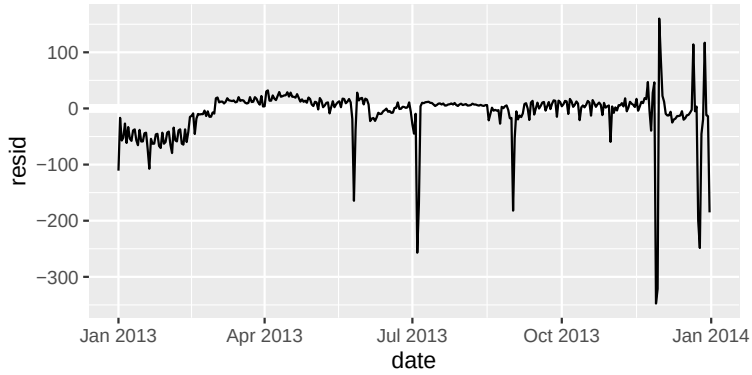
(The weekday labels depend on the locale: on a Korean system `wday(label = TRUE)` returns the Korean abbreviation instead, as on the course slides, so the filter must use that label.) Allowing the weekday effect to vary by term, `n ~ wday * term`, improves the fit:

```
mod1 <- lm(n ~ wday, data = daily)
mod2 <- lm(n ~ wday * term, data = daily)
daily %>%
  gather_residuals(without_term = mod1, with_term = mod2) %>%
  ggplot(aes(date, resid, colour = model)) +
  geom_line(alpha = 0.75)
```



The model fits the *mean* of each weekday–term cell, which is pulled by outliers (the holidays). A robust fit, `MASS::rlm()`, downweights outliers and gives a model that captures the regular pattern better, leaving the holidays clearly visible as large residuals:

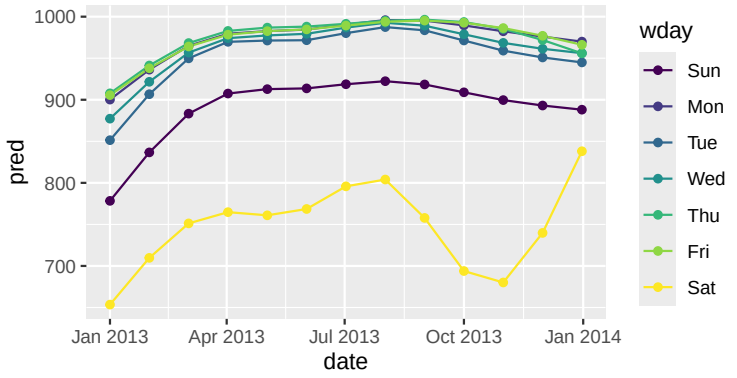
```
mod3 <- MASS::rlm(n ~ wday * term, data = daily)
daily %>%
  add_residuals(mod3, "resid") %>%
  ggplot(aes(date, resid)) +
  geom_hline(yintercept = 0, linewidth = 2, colour = "white") +
  geom_line()
```



14.3.3 Time of year: an alternative approach

Instead of encoding knowledge explicitly as terms, a more flexible model can let the data speak. A natural spline in date, interacting with weekday, fits a smooth seasonal curve for each weekday:

```
library(splines)
mod <- MASS::rlm(n ~ wday * ns(date, 5), data = daily)
daily %>%
  data_grid(wday, date = seq_range(date, n = 13)) %>%
  add_predictions(mod) %>%
  ggplot(aes(date, pred, colour = wday)) +
  geom_line() + geom_point()
```



The strong pattern in Saturday flights appears again without having been specified, which is reassuring: two different approaches pointing to the same signal.

Explicit versus flexible models [Supplement]

The `term` model encodes a hypothesis (school holidays) in three parameters per weekday and is easy to interpret, but its break dates were read off the plot, so it should not be tested on the same data. The spline model assumes only smoothness and uses more parameters; it is harder to interpret but less dependent on guesses. Agreement between them is evidence that the Saturday pattern is real rather than an artefact of either choice.

14.4 Selected exercises

Motivation. Model building rests on transformations and on choosing how many parameters to spend. The exercises make precise what a log–log line says about prices, what back-transformed predictions estimate, and why a model with a parameter for every cell is not a better model.

Exercise 14.1 (R4DS Exercise 24.2.3.2)

If $\log_2(\text{price}) = a_0 + a_1 \log_2(\text{carat})$, what does that say about the relationship between price and carat?

Solution.

Step 1. Exponentiating with base 2, price $= 2^{a_0}\,\text{carat}^{a_1}$: a power law. The base of the logarithm does not matter for a_1, since $\log_2 u = \log u / \log 2$ rescales both sides by the same factor.

Step 2. Multiplying carat by a factor c multiplies the predicted price by c^{a_1}, whatever the starting size: doubling the weight multiplies the price by 2^{a_1}. For a small relative change, $d\log(\text{price})/d\log(\text{carat}) = a_1$, so a 1% heavier diamond costs about a_1% more (a_1 is the elasticity).

Step 3. If $a_1 > 1$, price per carat, $2^{a_0}\text{carat}^{a_1-1}$, increases with size: large diamonds are disproportionately expensive. A residual of r on the $\log_2$ scale means the price is 2^r times the prediction, which is how the residual plots of this lecture were read. ■

Exercise 14.2 (back-transforming predictions)

Suppose $\log_2(\text{price}) = \mu(x) + \epsilon$ with $\epsilon \sim N(0, \sigma^2)$ independent of x. Show that $2^{\mu(x)}$, the back-transformed prediction used in the lecture, is the conditional *median* of price, and that the conditional mean is $2^{\mu(x)} \exp\{(\log 2)^2 \sigma^2 / 2\}$.

Solution [Supplement].

Step 1. The map $u \mapsto 2^u$ is strictly increasing, so $P(\text{price} \le 2^{\mu(x)} \mid x) = P(\epsilon \le 0) = \frac{1}{2}$: $2^{\mu(x)}$ is the conditional median.

Step 2. For the mean we need $\mathbb{E}[2^\epsilon] = \mathbb{E}[e^{c\epsilon}]$ with $c = \log 2$. Completing the square in the exponent,

$$\begin{aligned}\mathbb{E}[e^{c\epsilon}] &= \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}\sigma} \exp\Big(cu - \frac{u^2}{2\sigma^2}\Big) du \\ &= e^{c^2\sigma^2/2} \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}\sigma} \exp\Big(-\frac{(u - c\sigma^2)^2}{2\sigma^2}\Big) du,\end{aligned}$$

and the last integral is 1 (a normal density with mean $c\sigma^2$).

Step 3. Hence $\mathbb{E}[\text{price} \mid x] = 2^{\mu(x)}\mathbb{E}[2^\epsilon] = 2^{\mu(x)} e^{(\log 2)^2\sigma^2/2}$, larger than the median. Back-transformed predictions systematically underestimate mean prices by this factor; for typical prices (the median) they are right, which is what the plots compare. ■

Exercise 14.3 (R4DS Exercises 24.3.5.5 and 24.3.5.6)

(a) What happens if you fit a day-of-week effect that varies by month, `n ~ wday * month`, with `month` a factor? Why is this not very helpful?
(b) What would you expect the model `n ~ wday + ns(date, 5)` to look like?

Solution.

(a)

Step 1. The design has a column space spanned by the indicators of the $7 \times 12 = 84$ weekday–month cells, so the model has 84 coefficients for 365 days. As in Exercise 13.2, least squares then fits each cell by its own mean, since the residual sum of squares separates over cells.

Step 2. Every cell contains only four or five days, so each fitted value is a mean of four or five counts. A single holiday in a cell moves its mean substantially, and the fit follows these accidents rather than a regular pattern: the model has little bias but high variance and generalises poorly. It also cannot express the actual structure, a Saturday effect that changes with school terms, more economically than one free parameter per cell, which is why `wday * term` with a handful of parameters is preferable.

(b) The model is additive: the same smooth seasonal curve with five degrees of freedom for every weekday, shifted up or down by a constant weekday effect. The predictions are seven parallel curves. They capture the overall seasonal trend but not the summer rise in Saturday flights, which needs the curve itself to differ between weekdays, as in `wday * ns(date, 5)`; the residuals for Saturdays would therefore still show the term pattern. ■